Program for testing the performance of a PC

Rares-Flaviu Gog, Group 30434

Assignment

"Structure of Computer Systems" - 3rd year Computer Science, Project 14: Program for testing the performance of a PC: The type of processor, frequency, memory size, transfer speed of a block of data, execution time of arithmetic and logical operations.

1. Project Description

The project will display general information about PC hardware such as CPU type, cores, threads, memory RAM and GPU names. Then, to test the performance, the project will have to run some tests to generate relevant information. To grasp the performance of a PC each test should run multiple times and make an average out of these test runs to arrive at predictable, useful conclusions.

These tests are:

1.1. Multithreaded Numerical Computation Test

This test evaluates the system's behaviour under computational workloads distributed to different amounts of threads. The program computes all prime numbers less than or equal to a given number N using 1,2,4,8,16,32,64 threads. Execution time is measured for each configuration to assess how the processing speed is affected.

1.2. Basic Arithmetic Performance Test

This test measures the processor's ability to perform simple arithmetic operations. Repeatedly executing a single operation, such as addition, in a timed loop will determine the number of operations that can be completed per second. The results reflect the processor's arithemtic performance.

1.3. Algorithmic Sorting Performance Test

This test compares the computational cost of different sorting algorithms, such as Bubble Sort, Quick Sort and Merge Sort. Each algorithm sorts the same data while execution time is measured. The results highlight how algorithmic complexity and data access affect processors execution time.

1.4. Combined Arithmetic and Logic Operations Test

This test simulates real CPU workload by executing a mix of arithmetic and logical operations within a large iterative loop. The total number of operations performed in a second is computed to get the arithmetic/logic performance. This mixed benchmark aims to provide a more realistic measure of general compuational efficiency.

1.5. Memory Transfer and Bandwidth Test

This test evaluates memory performance by copying large data blocks using various buffer sizes. By varying the buffer size, the test aims to simulate the transition between cache levels and main memory, measuring data transfer rates. The results indicate the memory bandwidth.

1.6. Disk Throughput and I/O Performance Test

This test measures disk performance by writing and reading large data. The duration of each operation is recorded to calculate disk throughput and active time. This test aims to benchmark the system's storage performance.

1.7. Floating Point Operations Per Second (FLOPS)

This test measures the processor's ability to perform floating point arithmetic operations, such as addition, multiplication and division. Executing many operations in a timed loop, the program aims to estimate the number of floating-point operations per second. The results provide an estimation of the CPU's floating-point unit performance.

1.8. Possible addition

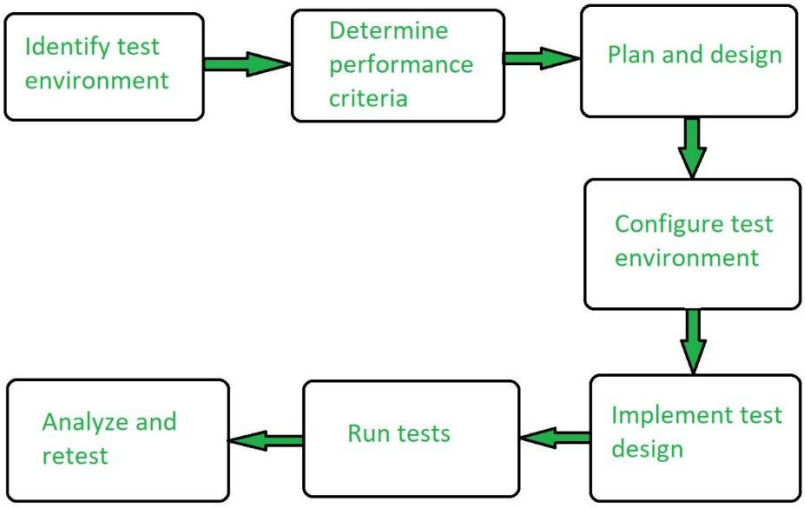
For each test the program has, it checks for power information in watts using the energy counters in the processor. This can be done by using the Intel Power Gadget API which provides real-time data on processor frequency and estimated processor power. It can also log frequency, power, energy and temperature data over time.

2. *Project Research*

2.1. Benchmark & Performance

Performance testing is a type of software testing that evaluates how a system behaves under specific workloads. These tests help predict how various components will respond under pressure. The aim is to identify improvements needed before a product is released, check for stability and reliability, detect performance bottlenecks and confirm that the system behaves as expected.

To conduct performance testing some steps must be followed:



- Testing Environment - Prepare the hardware, software and tools needed for testing.
- Performance Criteria - Determine what performance aspects to measure.
- Plan & Design - Design test cases that reflect real usage and define what data to collect.
- Configuration - Set up monitoring and measurement utilities.
- Run Tests - Execute the planned tests and record all performance data.
- Analyze Results - Review outcomes, identify bottlenecks and adjust parameters.
- Refine & Repeat - Re-run tests as needed until stable, reliable performance is achieved.

2.2. System Componenets

- *CPU* - Handles arithmetic, logic and control operations. CPU benchmarks include integer operations, floating-point calculations and multithreading tests to measure how efficiently the processor handles both simple and complex workloads.

- *Memory (RAM)* - Determines how quickly data can be accessed and transferred between memory and CPU. Memory benchmarks evaluate bandwidth and latency, using buffer copy operations to simulate data transfers.

- *Storage (Disk)* - Determines read/write speed and data transfer rates. Disk benchmarks use sequential and random I/O operations to measure throughput and identify the overall performance.

Benchmarking these components helps us understand the overall system's behaviour, reveals potential bottlenecks and ensures that the computer performs optimally for its intended tasks.

2.3. Benchmark Tests Information

2.3.1. Multithreaded Numerical Computation Test

This test evaluates the CPU's performance in handling parallel computational workloads by implementing the Sieve of Eratosthenes algorithm, to find all prime numbers less than or equal to a given number N. The sieve is executed using a different number of threads (1, 2, 4, 8, 16, 32, 64) while testing. The execution time for each configuration is measured. The goal is to analyze how computation speed scales with increasing thread counts, assessing the multi-core utilization, thread scheduling efficiency and memory bandwidth limitations of the system. The Sieve of Eratosthenes is used for this purpose because it performs repetitive arithmetic operations and memory marking in arrays, causing many cache reads and writes, making it a balanced algorithm to run tests on. This numerical benchmark highlights how performance improves up to the number of physical cores and then reacheas a limit where using many threads is rather meaningless. The test results show the CPU's scalability and efficiency in multithreaded cases.

2.3.2. Basic Arithmetic Performance Test

This test measures the processor's performance in executing simple arithmetic operations such as addition. The goal is to determine how many basic operations can be performed per second, representing the CPU's integer arithmetic throughput. Such benchmarks isolate specific CPU functions, such as the Arithmetic Logic Unit and it reveals how instruction execution time relates to clock frequency.

For this test doing one additon take a short amount of time due to the CPU's high performance. To solve this the program will loop milions (or more) additons per second while maintaining measurable and stable execution. It will also use keyword "volatile" for the variables that are used in the additon, so the compiler doesn't optimize the loop.

This provides a baseline for CPU raw speed in non-floating-point operations.

2.3.3. Algorithmic Sorting Performance Test

Various algorithms can be used to determine CPU efficiency for different sorting methods. This test runs these algorithms: Bubble Sort, Quick Sort and Merge Sort, on identical data, measuring their execution times. The comparison highlights how algorithm complexity and memory access patterns influence performance.

Such tests show how well a CPU and memory hierarchy handle branching, cache usage and large data movements, making it useful for evaluating both processing and data management efficiency.

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping the adjacent elements if they are in the wrong order. This algorithm is not suitable for large data sets as its average and worst-case time complexity are quite high**. ~GeeksForGeeks**

Complexity: O(n) -> O(n^2) Space: O(1)

QuickSort is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array. **~GeeksForGeeks**

Complexity: O(n log n) -> O(n^2) Space: O(log n)

Merge sort is a popular sorting algorithm known for its efficiency and stability. It follows the Divide and Conquer approach. It works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array. **~GeeksForGeeks**

Complexity: O(n log n) Space:O(n)

2.3.4. Combined Arithmetic and Logic Operations Test

This test performs a mixture of arithmetic (+, -, *, %) and logical (&, |, ^) operations inside a large loop to simulate a real-world workload. The objective is to measure combined instruction throughput, get a balanced representation of CPU usage compared to single-operation tests. This should be considered important since the CPU doesn't handle only arithmetic operations when handling work, it usually has a combination of arithmetic and logic operations, hence this test is present in this project.

The program will have a few variables that are used inside a loop that iterates many times. Inside, with the variables, it will perforam logic and arithemtic operations. The results indicate how efficiently the processor can handle diverse instruction types simultaneously, reflecting its overall computational efficiency.

2.3.5. Memory Transfer and Bandwidth Test

This test assesses memory speed and cache behaviour when handling large data blocks with varying buffer sizes. The rate of data transfer between memory and CPU is limited by cache size and hierarchy (L1, L2, L3, main memory). Smaller buffer sizes fit within the fastest cache and show higher speeds, while larger sizes gradually reveal slower transfers as they exceed cache capacity.

For this test the program will use the STREAM benchmark method for effective main memory bandwidth measurement. It will measure the time of four loops that operate on large arrays that don't fit in the CPU cache. Each loop will perform different things (copy, scale, add, triad). Triad means that it will perform copy, add and scale.

The program will also use memcpy() to test library-level block transfer speeds over increasing buffer sizes to visualize cache-level transitions.

These methods help to identify how efficiently the system works with large amounts of data.

Memcpy() → This command copies a data block from one memory location into another. This can be used to measure transfer speed and evaluate bandwidth performance.

"**L1 or Level 1 Cache:** It is the first level of cache memory that is present inside the processor. It is present in a small amount inside every core of the processor separately. The size of this memory ranges from 2KB to 64 KB.

L2 or Level 2 Cache: It is the second level of cache memory that may present inside or outside the CPU. If not present inside the core, it can be shared between two cores depending upon the architecture and is connected to a processor with the high-speed bus. The size of memory ranges from 256 KB to 512 KB.

L3 or Level 3 Cache: It is the third level of cache memory that is present outside the CPU and is shared by all the cores of the CPU. Some high processors may have this cache. This cache is used to increase the performance of the L2 and L1 cache. The size of this memory ranges from 1 MB to 8MB." **~GeeksForGeeks**

2.3.6. Disk Throughput and I/O Performance Test

This test measures disk read and write speed by using large data. Execution time is used to calculate throughput and active disk time. Disk performance depends on block size, queue depth and whether the user is using an HDD or SSD.

The algorithm for this test will prepare a test file filling it with dummy data, seqeuntially write the dummy data inside another file, sequentailly read the data from that file and then compute the throughput which is:

$$Throughput\ \left(\frac{MB}{sec}\right) = \frac{File\ Size}{Time}$$

This benchmark reveals the storage system's ability to handle sequential operations, simulating real scenarios like file copying or large data storage.

The test will use:

- write() → This command writes data from memory in a file. This can be used to test disk writing throughput and assess efficiency.
- read() → This command reads data from a file into memory. This can be used to test disk read speed and eavaluate performance.
- Fwrite() → A standard library function used to write a block of data from the program into a file.
- Fread() → A standard library function used to read the given amount of data from a file stream.

2.3.7. Floating-Point Operations Per Second (FLOPS) Test

The floating-point operations test measures how many floating-point operations (additions, multiplications, divisions or trigonometric calculations) the processor can execute in one second. Floating-point arithmetic is handled by the Floating-Point Unit (FPU), which operates on real numbers with decimal components. This metric shows computational performance, which is important in fields such as science, engineering and graphics.

For this test the program will follow the GEMM benchmark. This benchmark has the following pattern:

- Takes as its only parameter a size N
- Allocates matrices A, B and C of size N x N
- Initialize the matrices A and B with random data
- Execute matrix multiplication (result stored in matrix C) in a loop
- Calculate performance (FLOPs = 2 x N^3 x iterations)

This benchmark aims to stress the FPU making it ideal for assesssing raw computation power.

2.4. Display System Information

The application will display system information on which the benchmark is executed. It will provide information about:

→CPU Model

→CPU cores and logical processors

→CPU base clock speed

→ GPU Model

→ Total RAM inside the PC

The information is retrieved using Windows API's along with other libraries.

For the CPU the app will use CPUID functions "__CPUID()", each returning 16 bytes of the full brand string, which when concatenated create the name. The pysical cores and logical processors are fetched using Windows API functions like "GetLogicalProcessor InformationEx()". Each structure with a "RelationProcessorCore" is equivalent to a physical core. The number of logical processors is determined by counting the bits set in the processor using "__POPCNT()". The base clock speed is fetched by using Windows Registry with functions like "RegOpenKeyEx()" and "RegQueryValueEx()". Windows stores the frequency in the hardware description system control processor. The GPU info is obtained from the DISPLAY_DEVICE structure. Duplicate entries are filitered to avoid listing the same GPU. For this we use functions like "EnumDisplayDevices()". To display RAM we use "GlobalMemoryStatusEx()" which provides detailed info about memory.

2.5. Timing Methods in Benchmarking

Measuring execution time is used in benchmarking because it allows the calculation of performance metrics such as operations per second, data throughput and latency. There are multiple ways for obtaining time measurements in C, such as using the standard library function clock(), the hardware instruction RDTSC or the collection <chrono>.

2.5.1. Using clock()

The clock() function measures the processor time consumed by the program. It returns the number of clock ticks (time that has passed) up to the moment when the function was called, which can be converted into seconds by dividing by CLOCKS_PER_SEC. It is portable across systems. It is suitable for longer-running tests such as memory transfers or disk operations.

However, its precision is limited to the system's clock resolution and has an overhead that can alter measurements. It has a type-unsafe API which means it does not distinguish between time points and time duration, so mixing values can be a problem when working with it.

2.5.2. Using RDTSC

RDTSC is an x86 assembly instruction that reads the CPU's internal time-stamp counter, which increments once every clock cycle. It is accurate with minimal overhead. It can be used for benchmarks.

To ensure accurate reading, it is commonly used together with the CPUID or RDTSCP instructions to prevent ordering errors in execution. The difference between two points in time (end - start) gives the number of CPU cycles needed, which can be converted into time using the processors frequency.

Even if it is precise, this method is hardware-specific and less portable across platforms.

2.5.3. Using <chrono>

The <chrono> library provides the programmer with high-precision, wall clock timing, that is portable and easy to use. There are three main types of clocks: sytem_clock, steady_clock and high resolution_clock.

- system_clock - represents the system-wide real time wall clock. It's affected by the system's time adjustments.
- steady_clock - represents a monotonically increasing clock that's not affected by changes to the system time.

- high_resolution_clock - is the clock with the shortest tick period available on the system.

It measures real elapssed time (not CPU time) with high precision (nanoseconds) on most systems, as such it is good for benchamrking operations varying in duration.

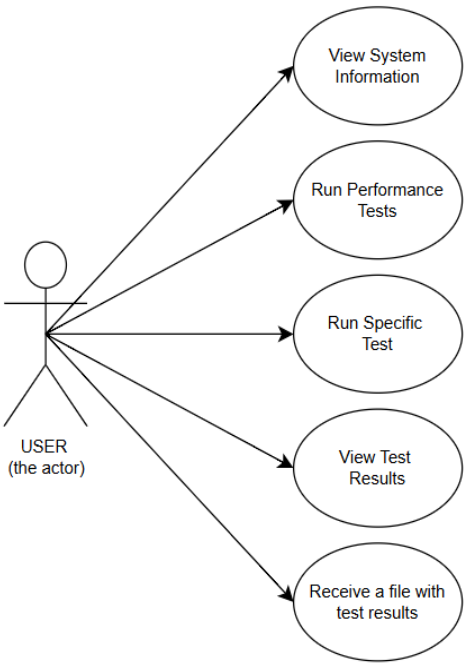
Conclusion: high_resolution_clock offers a perfect balance between precision, portability and ease of implementation, as such, this option will be used for all performance tests.

3.Analysis & Design

3.1. Analysis

This chapter analyzes the purpose and functionality of the PC Performance Testing program, identifying the system's requirements, main components and user interactions that define its overall architecture and behavior.

3.1.1. Diagrams



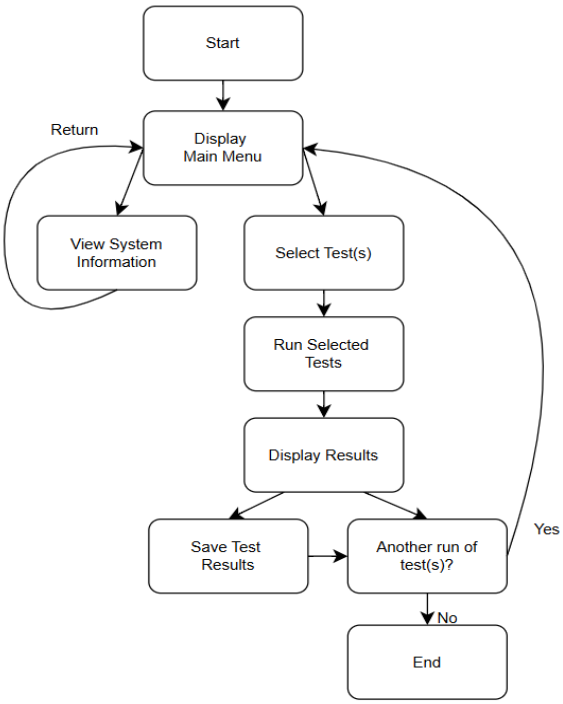


Fig.1. Use Case Diagram

Fig.2. Activity Diagram

The **Use Case Diagram** above shows how the user interacts with the Performance Testing Program.

Primary actor: USER

Use cases:

1.View System Information: Display the general information about the components the tests will run on.

2. Run Performance Tests: Run ALL tests available, one at a time.

3. Run Specific Test: Run one or more tests if the user wants specific benchmarks.

4.View Test Results: Shows the results obtained of the tests that were run.

5. Receive a file with test results: Save benchmark results into a file for later analysis.

The **USER** is the only actor in this system. He is the one triggering all program operations thorugh a simple interface.

The **Activity Diagram** above (Fig.2.) shows the workflow of the program, the steps that take place from start to finish.

Description:

1.The program shows the main screen. Here the user can either see system information or proceed to run the performance tests.

2.If the user chooses to view system information, the program will display hardware details, then return to main menu.

3.If the user selects to run tests, the program will display available tests and will let the user choose what test to run (one or more).

4.The selected tests are executed one at a time, duing which the program automatically measures and records the benchmark data.

5.After the tests are done, the user will see a screen with the results displayed. Here the user has the posibility to save the current test results in a file.

6.The user is asked if they want to run tests again, if not the program will end, otherwise the program will return to the main menu.

3.1.2. Functional & Non-Functional Requirements

Functional Requirements describe what the system must do, the feautures it provides to the user. These are:

- The system will display basic hardware information.
- The system will permit the user to select and run one or more performance tests.
- The system will perform benchmarks for CPU arithmetic, floating-point operations, memory bandwidth and disk throughput.
- The system will automatically monitor and record execution time during each test.
- The system will run each test multiple times for output accuracy.
- The system will display the test results to the users after completion.
- The system will enable the user to save or export the test results to a file for later analysis.
- The system will permit rerunning tests without restarting the application.

Non-Functional Requirements describe system qualities and constraints that affect performance and usability. These are:

- Each benchmark should be executed efficiently and complete within a reasonable time depending on the system's specifications.
- Time measurements must be precise, using functions the <chrono> library with high_resolution_clock to ensure reliable results.
- The program should run on common operating systems that support standard C libraries.
- The interface must be clear and simple, allowing users with basic technical knowledge to navigate and interpret results.
- The program must handle invalid input or failed tests without crashing.
- The program will be a Windows benchmarking app.

3.2. Design

3.2.1. Black-box Diagram

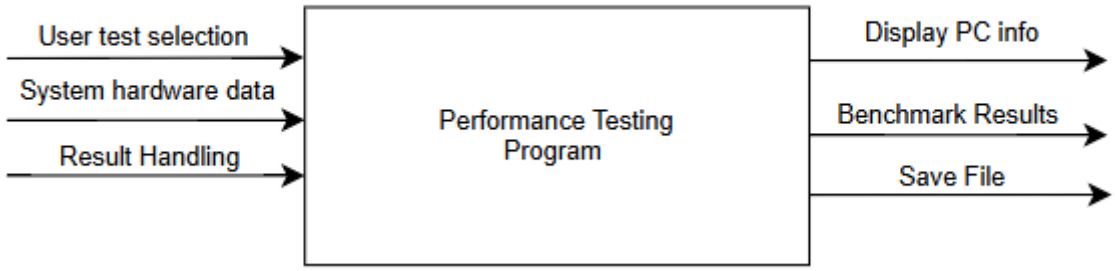


Fig.3. Black-box Diagram

The **Black-box Diagram** shows how the system receives the three main types of input (user commands, system hardware information, result management). Based on these inputs, the program will execute and will have three different types of output (the system information to display, benchmarking results to display and a file in which the benchmark tests are saved).

This representation aims to show the implementation details at an external level.

3.2.2. Folder Diagram

The **Structure Diagram** shows the internal organization of the source files for this project. Each file implements a specific functionality. This allows the project to be easly maintained

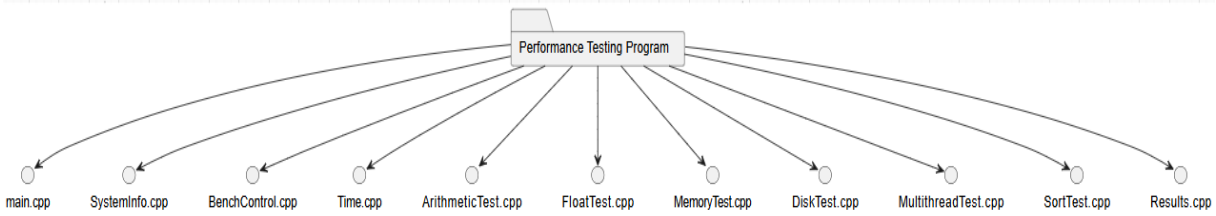


Fig.4. Folder Diagram

Description:

- main.cpp - The entry point of the program. Manages user input and manages the main menu.
- SystemInfo.cpp - Retrieves and displays hardware information such as CPU model, memory capacity and disk data.
- BenchControl.cpp - Acts as the central controller for running tests. Handles test selection.

- Time.cpp - Provides timing utilities using < chrono > collection with high_resolution_clock to measure execution duration for each benchmark.
- ArithmeticTest.cpp - Implements basic arithmetic and logic operation benchmarks.
- FloatTest.cpp - Executes floating-point operation tests to determine FLOPS performance.
- MemoryTest.cpp - Evaluates memory transfer speed and cache performance using memcpy() and STREAM.
- DiskTest.cpp - Measures disk read and write throughput using fread()/fwrite() and read()/write().
- SortTest.cpp - Runs sorting algorithms to analyze CPU efficiency on them.
- MultithreadTest.cpp - Runs the multithreading test to test CPU threads efficiency.
- Results.cpp - Handles displaying results on screen and saving them to a file for later analysis.

In addition, a file for Intel Power Gadget can be added to monitor other feautures of the CPU such as power consumption.

3.2.3. Architecture Diagram

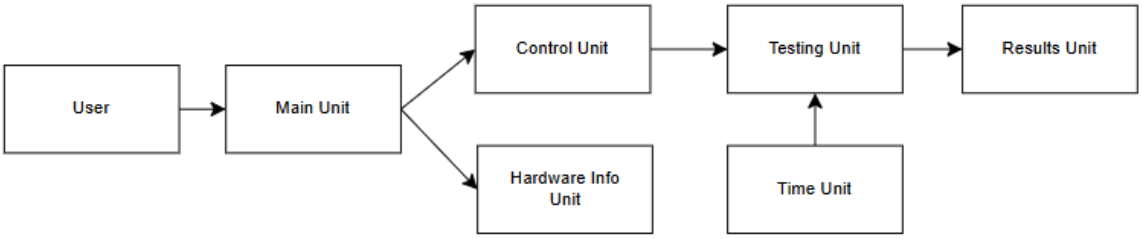


Fig.5 Achitecture Diagram

The **Architecture Diagram** above shows how elements "communicate" with eachother. The user interacts with the program through the Main Unit (main.cpp), which coordinates the program's activities. The Main Unit handles the Control Unit (BenchControl.cpp) which manages test execution, while Hardware Info Unit (SystemInfo.cpp) retrieves general system specifications. The Testing Unit ([…]Test.cpp) executes the selected benchmark tests with the timing data provided by the Time Unit (Time.cpp). The data is passed to the Results Unit (Results.cpp) which handles file saving and shows the results on screen.

3.2.4. GUI Design

The GUI is created in Python and it used Tkinter to provide an intuitive control UI for the user and an easy way to visualize the results of the C++ program. The app will require the executable file location to run (EXE_PATH).

→Tkinter is used for the layout, buttons, text areas and windows.

→Subprocess is used to execute the C++ executable of the benchmark app.

→Threading is used to prevent the GUI from freezing while running the tests.

→Matplotlib is used to generate and display a graph inside the GUI related to a specific test.

The GUI has buttons for viewing system info, run all tests, run a specific test which is selected from a drop-down menu, save test results button and two buttons to display graphs for specific tests. Under the button the GUI has a test display box. While running the tests or other stuff that requires time, the program will have a pop-up that displays "Loading…".

The Python code will use the C++ executable and use command-line arguments such as "--system" to do specific tasks implemented in the code. The C++ code has special lines in the tests that have graphs so the Python code can recognize them and generate the graph related to the results.

4.Validation & Tests

4.1 Short Tutorial

First step is to check that the EXE_PATH in the GUI.py file is the same with the .exe file of the C++ program. If that is ok, then the below window will pop-up.

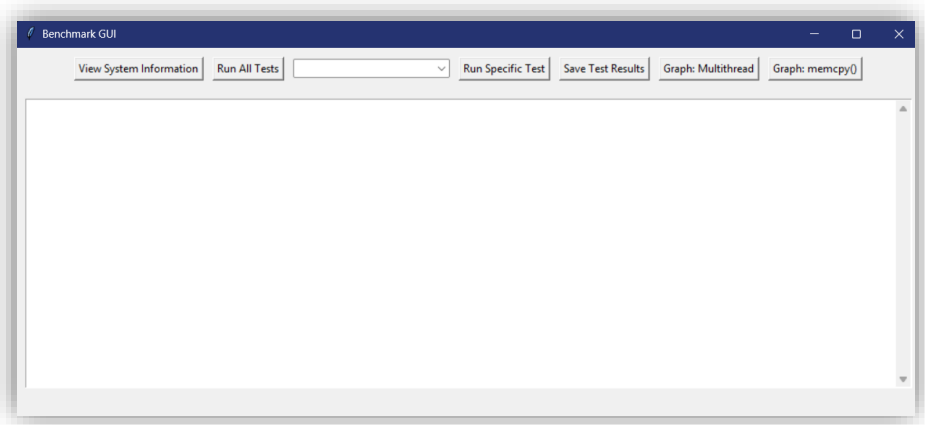


Fig.6. Main Menu

Each button has an intuitive description on them so I will not explain each one.
A special case is the Run Specific Test, which requires the user to select a test, just like in the image below:

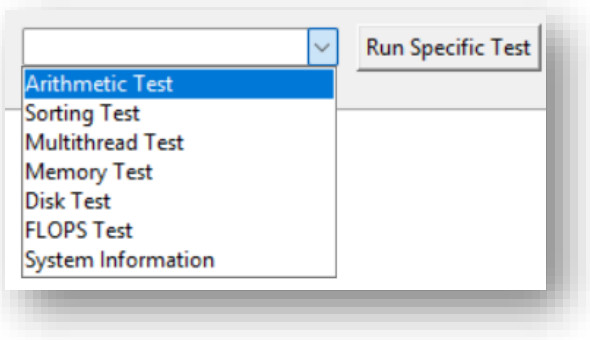


Fig.7. Drop-Down Selector

The pop-up for running specific tasks that require the user to wait is the following:

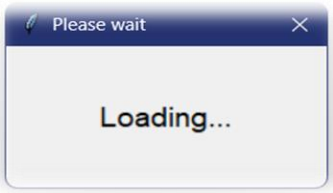
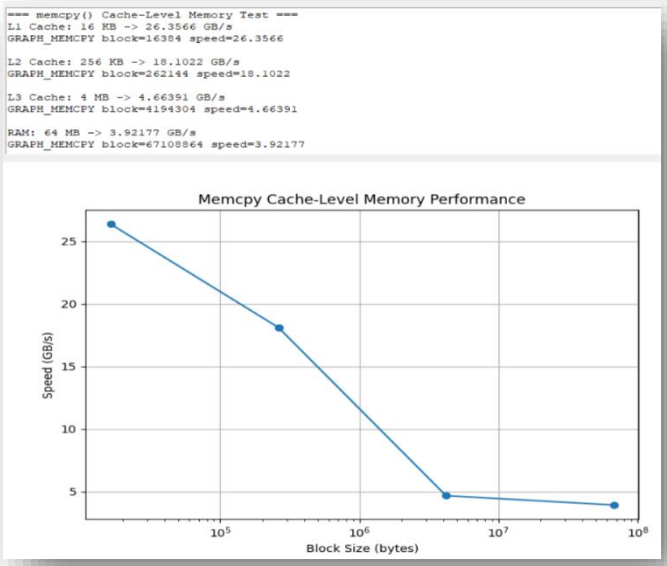


Fig.8. Pop-up Window while running tests



After running a test to generate a graph, this is what the app should display:

Here the results are displayed along with the graph. This example shows the cache memory level test results on a graph.

Fig.9. Graph Display for memory test

4.2. Validating Results

The below results are from a previous run of the program.

4.2.1. Basic Arithmetic Test Results

Results:

“

=== Basic Arithmetic Performance Test ===

Runs: 10

Average Result: 2.13 GADDs/sec

“

The result obtained from the test that performs only integer additions in a loop with no memory acces, no branching, no function calls has a good value. The result is limited, in the most part, only by the CPU clock frequency and instruction throughput. Over 10 runs this is an expected value. This result is close to the CPU clock frequency of my PC which I will talk about below in a different segment.

4.2.2. Combined Arithmetic Test Results

Results:

“

=== Combined Arithmetic and Logic Operations Test ===

Runs: 10

Average Result: 3.10 GOPs/sec

“

The result above is a mix of arithemtic and logical operations which run on different execution units inside the CPU. Compared to the pure addition test, this one has better parallelism between instructions, so it is more balanced, which leads to the higher value displayed. The expected value should be greater than GADDs/sec all the time.

4.2.3. Disk Test Results

Results:

“

=== Disk Performance Test ===

=== WRITE/READ Test ===

Runs: 10

Average write speed: 227.40 MB/s

Average read speed: 340.03 MB/s

=== FREAD/FWRITE Test ===

Runs: 10

Average write speed: 2181.03 MB/s

Average read speed: 2096.55 MB/s

”

For these tests that run on 200 MB file size using 1 MB blocks to minimize overhead we can see a big difference between FREAD/FWRITE and READ/WRITE. Fread and fwrite are faster than read and write because they automatically use internal buffers, so the program accesses the disk fewer times, which reduces overhead and results in better disk performance. Also read and write use streams while fread and fwrite use C Standard I/O. Since the fread/fwrite gets internally optimized it is faster and that can be seen in the results as such they are correct.

4.2.4. Memory Test Results

Results:

“

=== Memory STREAM Test ===

Runs: 10

Copy: 2.00 GB/sec

Scale: 2.02 GB/sec

Add: 1.61 GB/sec

Triad: 1.36 GB/sec

=== memcpy() Cache-Level Memory Test ===

L1 Cache: 16 KB -> 126.67 GB/sec

L2 Cache: 256 KB -> 44.28 GB/sec

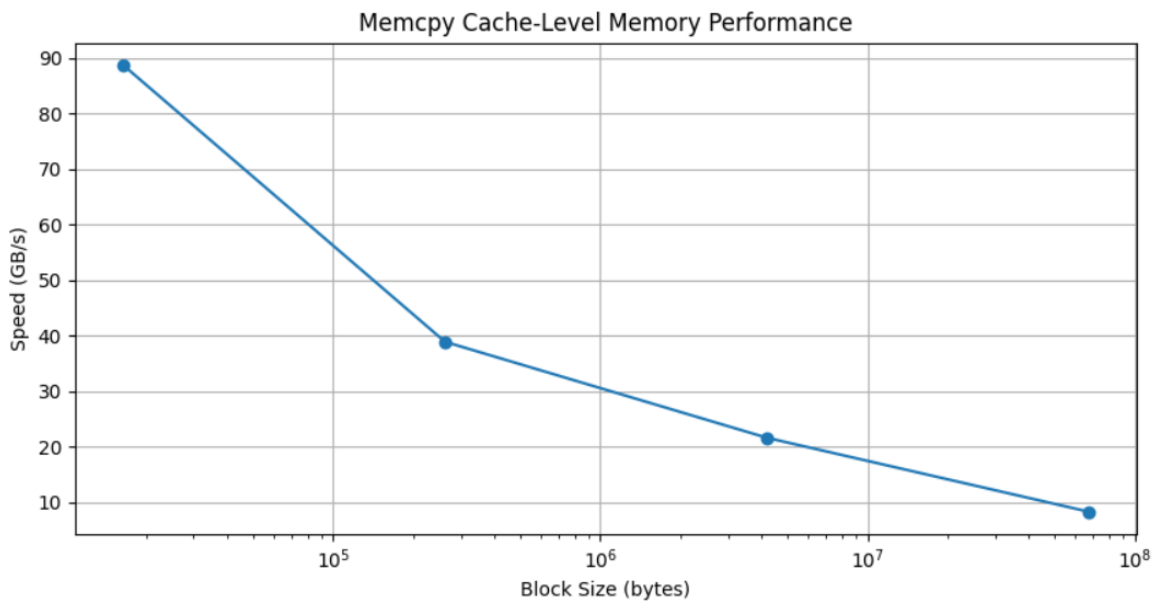
L3 Cache: 4 MB -> 23.01 GB/sec

RAM: 64 MB -> 8.71 GB/sec

”

For the STREAM test copy has 2 memory accesses, scale has arithmetic and memory access, add has 3 memory accesses and traid has all of them together. More memory accesses means a lower bandwidth. The memory used per iteration is: COPY (16 bytes), SCALE (16 bytes), ADD (24 bytes) and TRIAD (24 bytes) This test measures main memory bandwidth and follows the STREAM model reseached.

For the Cache-Level test we can see that L1 is fast as it is the closest to the CPU, then as the size gets larger and we move to L2 then to L3 we can see the speed lowering until we reach RAM where the speed stabilizes. Using fixed sizes ensures the data is within the cache level that is tested. Since it follows the theory behind the test the values are the expected ones.



4.2.5. Multithreading Test Results

Results:

“

=== Multithreaded Numerical Computation Test ===

Running test with 1 threads

Runs: 10

Average time: 15.01 sec

Running test with 2 threads

Runs: 10

Average time: 7.29 sec

Running test with 4 threads

Runs: 10

Average time: 4.06 sec

Running test with 8 threads

Runs: 10

Average time: 1.68 sec

Running test with 16 threads

Runs: 10

Average time: 1.14 sec

Running test with 32 threads

Runs: 10

Average time: 1.14 sec

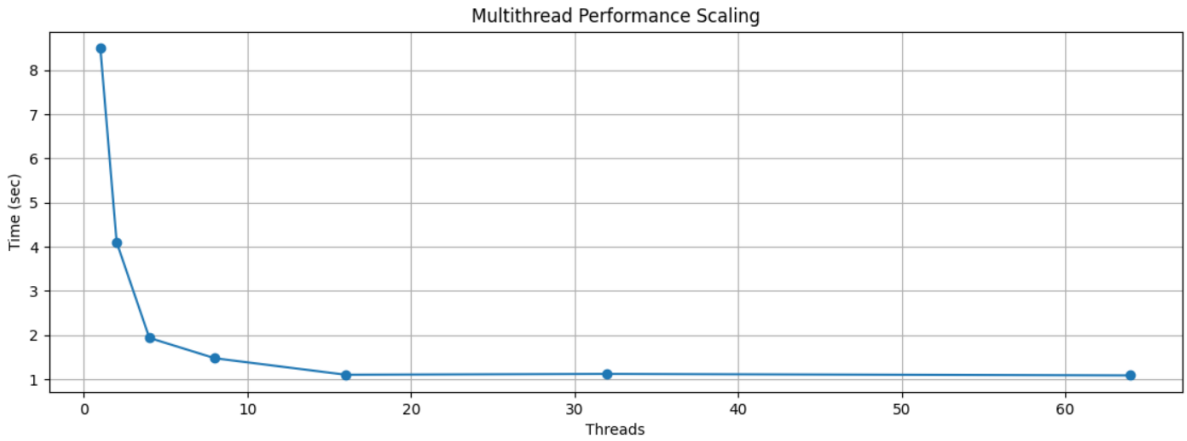
Running test with 64 threads

Runs: 10

Average time: 1.14 sec

“

The behaviour of this test is correct since from 1 to 8 threads the speed is growing due to effective workload splitting, but after 16 threads the CPU reaches the limit and memory bandwidth becomes a bottleneck and threads no longer make the algorithm more efficient. The test reaches a so called “plateau” after hardware limitations, so the results are what is expected.



4.2.6. Sorting Algorithms Test Results

Results:

“

=== Sorting Performance Test ===

Runs: 10

Buffer size: 30000

Bubble Sort: 16.78 sec

Quick Sort: 0.02 sec

Merge Sort: 0.19 sec

“

For Bubble Sort the algorithm compares every element with every other element most of the time. For such a big number (30000) the amount of operations is high. Since there are no optimizations there might not early exist, so we get the worst-case behaviour. It doesn't use extra memory, so it has constant space. This algorithm is expected to be slow and that can be seen in the results.

For Quick Sort the algorithm uses divide-and-conquer method, so the cache access and low memory overhead allow it to run much faster. The algorithm is expected to run this fast for the input size given.

For Merge Sort the algorithm is like Quicksort in time but is a bit slower because it uses extra memory allocation and it has more data movement. So, the result is expected to be slower than quick sort but still fast, which is exactly what the results show.

4.2.7. Floating Point Test Results

Results:

"

=== Floating Point Performance Test ===

Runs: 10

Matrix size: 300 x 300

Average performance: 0.09 Gflops/sec

"

The FLOPS test measures how fast the CPU can perform floating-point operations by multiplying two matrices using a simple triple for loop. Each matrix multiplication involves many repeated multiply and add operations, which makes it a good way to stress the CPU's floating-point units. The relatively low GFlops value is expected because the implementation is not optimized (no cache blocking, no parallelism) and uses standard C++ loops, so the result mainly reflects raw CPU computation ability rather than highly optimized numerical performance. As such, the result is the expected one and is correct.

4.2.8. System Information

For my PC the below information is displayed correctly:

Displayed:

“

=== SYSTEM INFORMATION ===

CPU: 12th Gen Intel(R) Core(TM) i5-12450H

Cores: 8

Logical processors: 12

Base speed: 2.496 GHz

GPU #0: Intel(R) UHD Graphics

GPU #1: NVIDIA GeForce RTX 3050 Ti Laptop GPU

Memory RAM: 31.7316 GB

”

5.Conclusion

This project provided a practical and structured way to analyze and understand the performance characteristics of a modern computer system. By implementing multiple benchmarks targeting the CPU, memory hierarchy, disk subsystem and multithreading, the project shows how different hardware components behave under specific workloads and why their performance differs.

Integrating all benchmarks into a Python GUI improved usability and visualization, allowing results to be collected, saved and plotted in an intuitive way. Overall, this project successfully uses theoretical concepts, offering a clear and realistic view of system performance and reinforcing the importance of knowing the hardware you work on.

6.Bibliography

- https://www.geeksforgeeks.org/software-testing/performance-testing-software-testing/
  ~opened: 10.10.2025
- https://en.wikipedia.org/wiki/Benchmark_(computing)
  ~opened: 10.10.2025
- https://en.wikipedia.org/wiki/Memory_bandwidth
  ~opened: 13.10.2025
- https://systemsarchitect.io/docs/requirements/estimates/compute/disk-throughput
  ~opened: 13.10.2025
- https://www.geeksforgeeks.org/computer-science-fundamentals/cache-memory/
  ~opened: 13.10.2025
- https://en.wikipedia.org/wiki/Floating_point_operations_per_second
  ~opened: 14.10.2025
- https://en.wikipedia.org/wiki/Floating-point_unit
  ~opened: 14.10.2025
- https://www.intel.com/content/www/us/en/developer/articles/training/using-the-intel-power-gadget-api-on-mac-os-x.html
  ~opened: 14.10.2025
- https://www.geeksforgeeks.org/cpp/chrono-in-c/
  ~opened. 10.11.2025
- https://documentation.sigma2.no/jobs/arm-perf/stream.html
  ~opened: 11.11.2025
- https://www.amd.com/en/developer/zen-software-studio/applications/spack/stream-benchmark.html
  ~opened: 11.11.2025
- https://cs.stackexchange.com/questions/9144/calculating-floating-point-operations-per-secondflops-and-integer-operations-p
  ~opened: 11.11.2025
- https://stackoverflow.com/questions/700570/how-can-i-write-a-program-which-can-test-throughput-of-disk
  ~opened 11.11.2025
- https://www.intel.com/content/www/us/en/developer/articles/technical/benchmarking-gemm-with-intel-mkl-and-blis-on-intel-processors.html
  ~opened 11.11.2025
- https://learn.microsoft.com/en-us/windows/win32/api/sysinfoapi/nf-sysinfoapi-getlogicalprocessorinformationex
  ~opened 20.12.2025
- https://learn.microsoft.com/en-us/cpp/intrinsics/cpuid-cpuidex?view=msvc-170
  ~opened 20.12.2025

- https://learn.microsoft.com/en-us/windows/win32/api/winuser/nf-winuser-enumdisplaydevicesa
  ~opened 20.12.2025
- https://learn.microsoft.com/en-us/windows/win32/sysinfo/registry-functions
  ~opened 20.12.2025
- https://learn.microsoft.com/en-us/windows/win32/api/sysinfoapi/nf-sysinfoapi-globalmemorystatusex
  ~opened 20.12.2025